Mechanic-shop Project

1- Manager

The manager can add new parts and determine its salary and all things that related to it like quantity…etc.
And he can add new repair task every repair task determine the parts he wants but not the quantity for example Brake Replacement this repair tasks wants Brake Fluid, Brake Pads
But here we won't determine the quantity
The manager can give one or more than Employee WorkOrder if they are free in that time he wants, employee and the manager can determine the quantity of the part in this moment and based on that the manager can release the invoice (Note: the price is based on the current hour so if we deal with customer with specific price then the price increase the price that the customer will pay is the first one will not increase)
The manager can add customers but the customers have to sign up first and enter their info and the vehicles they own

2- Employee

Employee can accept the work order that the manager gave it to them and can change the status for the work order from Scheduled to in progress to completed or to canceled

3- Customer

Customer must sign up before they to the shop
Customer can determine the time that he wants to come (Optional in the future not mvp)
Customer can track the status of his vehicle and receive the notifications (Optional in the future not mvp)

Detailed Role-Based Functionality

1. Manager (Employee with elevated privileges)

- **Part Management:**
  - Add/Edit/Delete Part records (Name, Description, Category, Supplier).
  - Manage **StockQuantity** (manual adjustment or via inventory system).
  - Manage Part Pricing: Update **CurrentCost** on **Part**. This automatically creates a new record in **PartPriceHistory** with **EffectiveFrom** set to now. *This ensures future invoices use the correct historical cost.*
- **Repair Task Management:**
  - Define standard **RepairTask**s (Name, EstimatedDuration, DefaultLaborCost).
  - Link **RepairTask**s to **Part**s using **RepairTaskPart** (specifying *which parts are typically required* for this task, without quantity).
- **Work Order Creation & Assignment:**
  - Create a new **WorkOrder** for a specific **Vehicle**.
  - Assign one or more **Employee**s to the **WorkOrder** via **WorkOrderEmployee** (specifying **HoursWorked** - initially estimated, later updated). *System should check employee availability (optional feature).*
  - Add specific **RepairTask**s to the **WorkOrder** via **WorkOrderRepairTask**. When added, the system captures the *current* **DefaultLaborCost** from the **RepairTask** and stores it as **LaborCostAtUse** in the linking table. *This locks in the labor rate at the time of assignment, preventing future price increases from affecting this order.*
  - Add specific **Part**s to the **WorkOrder** via **WorkOrderPart**. When added, the system captures the *current* **CurrentCost** from the **Part** and stores it as **UnitPriceAtUse** in the linking table. *This locks in the part cost at the time of use, ensuring accurate invoicing even if prices change later.* The **QuantityUsed** is also recorded here.
- **Invoice Generation:**
  - Trigger invoice creation when a **WorkOrder** is marked as **Completed**.
  - Calculate **Subtotal** = Sum of (**QuantityUsed** * **UnitPriceAtUse**) for all **WorkOrderPart** + Sum of (**LaborCostAtUse** for each **WorkOrderRepairTask**).
  - Apply **Discount** (if any).
  - Calculate **TaxAmount** = **Subtotal** * **TaxRate**.
  - Set **TotalAmount** = **Subtotal** - **Discount** + **TaxAmount**.
  - Set **PaymentStatus** to **Pending** and **IssuedAt** to current datetime.
  - Critical Rule: Prices (**UnitPriceAtUse**, **LaborCostAtUse**) are captured at the time of adding the item to the WorkOrder, not at invoice generation. This protects against price fluctuations and matches your requirement: "*the price increase... will not increase*".

2. Employee (Mechanic / Laborer)

- **Work Order Management:**
  - View assigned **WorkOrder**s (filtered by their **EmployeeId** in **WorkOrderEmployee**).
  - Update the **State** of a **WorkOrder** they are assigned to:
    - **Scheduled -> InProgress** (when starting work).
    - **InProgress -> Completed** (when finished).
    - **InProgress -> Canceled** (if work cannot be completed, requires manager approval?).
  - **Record Hours Worked:** Update the **HoursWorked** field in **WorkOrderEmployee** for their assignment. This directly impacts the final labor cost calculation on the invoice.
- **Part Usage Tracking:**
  - Record the actual **QuantityUsed** for **Part**s in **WorkOrderPart** as they are consumed during the repair. *This updates the **StockQuantity** in the **Part** table automatically.*

3. Customer

- **Account Management:**
  - Register/Login via the **User** system.
  - Manage personal profile (linked to **Customer** record).
- **Vehicle Management:**
  - Add/Edit/Delete **Vehicle** records associated with their account.
- **Service Request & Tracking:**
  - Initiate a service request (creates a **WorkOrder** or **Appointment**).
  - Track Status: View the current **State** of their **WorkOrder** (Scheduled, InProgress, Completed, Canceled).
  - View Invoice: Access the generated **Invoice** for completed **WorkOrder**s.
  - *(Future)* Schedule Appointments: Book a specific time slot via the **Appointment** system.
  - *(Future)* Notifications: Receive alerts (email/push) for status changes or appointment reminders.

Key Business Logic & Rule

1. **Price Locking:** All costs (**UnitPriceAtUse**, **LaborCostAtUse**) are captured and frozen at the moment a **Part** or **RepairTask** is added to a **WorkOrder**. This ensures billing accuracy regardless of future price changes.

2. **Inventory Management: StockQuantity** in **Part** is decremented when **QuantityUsed** is recorded in **WorkOrderPart** during the repair process.
3. **Labor Cost Calculation:** The total labor cost for a **WorkOrder** is the sum of (**HoursWorked** * **HourlyRate** at the time of work) for each assigned **Employee**. The **HourlyRate** used should be retrieved from **EmployeeSalaryHistory** based on the **WorkOrder**'s **StartAt** date to ensure historical accuracy. *Note: Your ERD doesn't explicitly link **WorkOrderEmployee.HoursWorked** to **EmployeeSalaryHistory.HourlyRate** directly; this logic needs to be implemented in the application layer.*
4. **Work Order State Transitions**: Defined workflow: **Scheduled** -> **InProgress** -> **Completed** or **Canceled**. Only **Employee**s can change the state from **Scheduled** to **InProgress** and onwards.
5. **Invoice Trigger:** Invoices are generated only when a **WorkOrder** reaches the **Completed** state. The **Invoice** record is linked 1:1 to the **WorkOrder**.

Workflow Summary

1. Customer Signs Up -> Creates **User** + **Customer** record.
2. Customer Adds Vehicle -> Creates **Vehicle** record linked to **Customer**.
3. Manager Creates Work Order -> Selects **Vehicle**, assigns **Employee**s, adds **RepairTask**s (capturing **LaborCostAtUse**), adds **Part**s (capturing **UnitPriceAtUse**).
4. Employee Starts Work -> Changes **WorkOrder.State** to **InProgress**, records **HoursWorked** and **QuantityUsed**.
5. Employee Completes Work -> Changes **WorkOrder.State** to **Completed**.
6. System Generates Invoice -> Calculates **Subtotal**, applies **Discount**, calculates **Tax**, sets **TotalAmount**, **PaymentStatus=Pending**, **IssuedAt=Now**.
7. Customer Views Status/Invoice -> Can see **WorkOrder.State** and access the **Invoice**.

Indexes

Index Name	Table	Index Type	Why This Index Was Created	Why This Type Was Used
IX_Vehicle_CustomerId	**Vehicle**	Non-Clustered Index	Speeds up retrieving all vehicles belonging to a customer. Frequently used in customer dashboard queries.	A non-clustered index is ideal because filtering is performed on **CustomerId** without changing the physical order of the table.
IX_WorkOrderEmployee_EmployeeId	**WorkOrderEmployee**	Non-Clustered Covering Index	Optimizes employee dashboard queries that retrieve assigned work orders by **EmployeeId**.	Non-clustered index allows fast filtering by employee, while **INCLUDE** columns reduce additional lookups and improve read performance.
IX_WorkOrder_VehicleId_State_StartAt	**WorkOrder**	Composite Non-Clustered Index	Improves filtering and sorting of work orders by vehicle, state, and start date.	Composite indexing is useful because queries commonly filter on multiple columns together.
IX_EmployeeSalaryHistory_EmployeeId_EffectiveFrom	**EmployeeSalaryHistory**	Composite Non-Clustered Covering Index	Optimizes historical salary lookup queries based on employee and date.	Composite index supports efficient range filtering and ordering by date. Included columns avoid extra table access.
IX_PartPriceHistory_PartId_Eff	**PartPriceHistory**	Composite Non-Clustered	Speeds up historical part	Composite indexing is

ectiveFrom		Covering Index	price retrieval used in price-locking logic.	effective for searching by **PartId** and latest effective date.
IX_WorkOrderPart_WorkOrderId	**WorkOrderPart**	Non-Clustered Covering Index	Optimizes invoice generation queries that retrieve parts used in a work order.	Covering index reduces logical reads because all required columns are stored directly in the index.
IX_WorkOrderPart_PartId	**WorkOrderPart**	Non-Clustered Index	Improves reverse lookups of work orders using a specific part.	Non-clustered index is suitable for filtering without changing table storage order.
IX_WorkOrderRepairTask_WorkOrderId	**WorkOrderRepairTask**	Non-Clustered Covering Index	Speeds up retrieval of repair tasks associated with a work order.	Covering index minimizes key lookups and improves query performance.
IX_RepairTaskPart_PartId	**RepairTaskPart**	Non-Clustered Index	Optimizes reverse lookup queries for repair tasks using a specific part.	Non-clustered index efficiently supports filtering on **PartId**.
IX_Appointment_CustomerId_ScheduledAt	**Appointment**	Composite Non-Clustered Index	Improves customer appointment history and scheduling queries ordered by date.	Composite indexing supports filtering by customer and sorting by appointment date efficiently.

Performance Statics Before & After indexes

Query	Before Indexes	After Indexes	Improvement
Employee Dashboard Query (**WorkOrderEmployee**)	3883 logical reads, 31 ms CPU, 48 ms elapsed	5 logical reads, 15 ms CPU, 6 ms elapsed	Massive reduction in reads and execution time
Customer Vehicles Query (**Vehicle**)	126 logical reads, 16 ms CPU, 1 ms elapsed	10 logical reads, 0 ms CPU, 0 ms elapsed	Faster lookup with fewer reads
Work Order Parts Query (**WorkOrderPart**)	3211 logical reads, 31 ms CPU, 30 ms elapsed	3 logical reads, 0 ms CPU, 0 ms elapsed	Extremely improved performance
Work Order Repair Tasks Query (**WorkOrderRepairTask**)	3 logical reads, 0 ms CPU, 0 ms elapsed	3 logical reads, 0 ms CPU, 0 ms elapsed	Already optimized by clustered PK
Employee Salary History Query	28 logical reads, 16 ms CPU, 1 ms elapsed	2 logical reads, 0 ms CPU, 0 ms elapsed	Improved historical lookup efficiency
Part Price History Query	2 logical reads, 0 ms CPU, 1 ms elapsed	2 logical reads, 0 ms CPU, 0 ms elapsed	Slight improvement
Appointment Lookup Query	Large table scan behavior	Efficient indexed lookup	Significant improvement in filtering and ordering

Overall, the indexes significantly reduced logical reads, CPU time, and elapsed time by converting expensive table scans into efficient index seeks, especially on large transactional tables such as **WorkOrderEmployee** and **WorkOrderPart**.

Views

View Name	Role	Why it is important	Main tables used
vw_ManagerDashboard	Manager	Gives a full overview of work orders, invoices, customers, vehicles, and employees in one place. Useful for daily management.	**WorkOrder**, **Invoice**, **Customer**, **Vehicle**, **Employee**
vw_WorkOrderDetails	Manager	Shows full work order information with assigned employees, repair tasks, and parts used. Very useful when creating or reviewing work orders.	**WorkOrder, WorkOrderEmployee, WorkOrderRepairTask, WorkOrderPart**
vw_InvoiceDetails	Manager	Makes invoice generation and invoice review easier because it already combines parts cost, labor cost, discount, tax, and total.	**Invoice, WorkOrder, WorkOrderPart, WorkOrderRepairTask**
vw_LowStockParts	Manager	Helps the manager quickly see which parts need restocking. Important for inventory control.	**Part**
vw_EmployeeWorkOrders	Employee	Shows only the work orders assigned to a specific employee. This is one of the most important views for the employee role.	**WorkOrder, WorkOrderEmployee, Vehicle, Customer**
vw_EmployeeWorkOrderTasks	Employee	Shows repair tasks assigned to the work order, so the employee can know	**WorkOrderRepairTask, RepairTask, WorkOrder**

		what to do.	
vw_EmployeeWorkOrderParts	Employee	Shows the parts used in the assigned work order, with quantity and locked price.	**WorkOrderPart, Part, WorkOrder**
vw_EmployeeSalaryCurrent	Employee / Manager	Useful for salary history lookup and historical payroll calculations.	**Employee, EmployeeSalaryHistory, User**
vw_CustomerProfile	Customer	Shows customer account information in one simple query.	**Customer, User**
vw_CustomerVehicles	Customer	Lets customers see all their vehicles easily. This is a core customer screen.	**Vehicle, Customer**
vw_CustomerWorkOrders	Customer	Lets customers track the status of their service requests and repair jobs. This is one of the most important customer views.	**WorkOrder, Vehicle, Customer**
vw_CustomerInvoices	Customer	Lets customers view only their own invoices. Very important for billing and transparency.	**Invoice, WorkOrder, Vehicle, Customer**
vw_CustomerAppointments	Customer	Useful for future appointment booking and tracking.	**Appointment, Customer, Vehicle**

Procedure

Stored Procedure Name	Why It Was Created
sp_CreateWorkOrder	Creates a new work order for a vehicle while validating that the vehicle exists and initializing the repair workflow.
sp_AddPartToWorkOrder	Adds parts to a work order, locks the current part price (**UnitPriceAtUse**), and updates inventory stock automatically.
sp_AddRepairTaskToWorkOrder	Adds repair tasks to a work order and locks the labor cost (**LaborCostAtUse**) at the time of assignment to prevent future price changes from affecting the invoice.
sp_UpdateWorkOrderState	Controls valid work order state transitions (**Scheduled → In_Progress → Completed/Cancelled**) and enforces workflow rules.
sp_RecordPartUsage	Updates the actual quantity of parts used during repair and synchronizes inventory stock quantities accordingly.
sp_GenerateInvoice	Generates invoices automatically for completed work orders by calculating subtotal, tax, discount, and total amount using locked prices and labor costs.